

WasteWise

Waste Management Cloud

Cloud Computing Case Study — AWS Implementation

Student: Swaraj Wattamwar

Program: B.Tech CSE — ITM Skill University

Cloud Platform: Amazon Web Services (AWS)

Github: <https://github.com/Swaraj2259/wastewise-dashboard>

1. The Problem

WasteWise is a growing waste management and recycling company operating across multiple regions. As the company expanded, their day-to-day operations became increasingly difficult to manage. Here is what their situation looked like before this project:

- Operations were managed through disconnected spreadsheets — no single source of truth
- Different teams in different regions had no shared system to track waste collections or vehicle status
- Reports were generated manually, causing delays and errors
- There was no way to monitor if systems were running or failing — issues were only discovered after users complained
- No automated backups — critical data could be lost permanently
- No security controls — anyone could access any system

The result was inefficiency, data loss risk, and zero visibility into operations. WasteWise needed a centralized, secure, and reliable platform to run their business.

2. The Solution — AWS Cloud Platform

We built a complete cloud infrastructure on Amazon Web Services (AWS) to replace all disconnected systems with one centralized platform. The solution covers six key areas:

- A private, secure network on AWS (VPC) that isolates all company resources
- A cloud server (EC2) running the web application and all services
- A managed database (RDS MySQL) storing all operational data securely
- Containerized services using Docker for consistent, portable deployments
- Automated monitoring via CloudWatch that alerts the team if anything goes wrong
- Shell scripts that automate backups, deployments, and health checks — no manual work needed

All infrastructure runs exclusively on AWS in the US East (N. Virginia) region, with resources spread across two Availability Zones for reliability.

AWS Service	Purpose	What it does in this project
VPC	Networking	Private isolated network for all WasteWise resources
EC2	Compute	Virtual server running Apache, Docker, and all scripts
RDS MySQL	Database	Managed DB storing waste collection and vehicle data
Security Groups	Firewall	Controls what traffic is allowed in/out of each resource
Internet Gateway	Internet	Connects the public subnet to the internet
CloudWatch	Monitoring	Tracks CPU, memory, disk, logs and sends alerts
ECR	Containers	Stores custom Docker images for WasteWise

3. Architecture Overview

The platform is built in layers. Internet traffic flows through an Internet Gateway into our private AWS network (VPC). Inside the VPC, resources are split between a public subnet (accessible from internet) and a private subnet (completely hidden).

The EC2 server sits in the public subnet — it runs Apache, Docker containers, and all automation scripts. The RDS database sits in the private subnet — it can only be accessed from the EC2 server, never from the internet. CloudWatch monitors everything.

AWS Region	US East — N. Virginia (us-east-1)
VPC Name	project-vpc-vpc
VPC CIDR	10.0.0.0/16
Public Subnet 1	10.0.1.0/24 — us-east-1a (EC2 lives here)
Public Subnet 2	10.0.2.0/24 — us-east-1b (Multi-AZ)
Private Subnet 1	10.0.128.0/20 — us-east-1a (RDS lives here)
Private Subnet 2	10.0.144.0/20 — us-east-1b (Multi-AZ)
Internet Gateway	project-vpc-igw — Attached and Active

4. Phase 1 — Network & Server Setup

4.1 Creating the VPC

The first step was creating our private network on AWS. A VPC is like having your own dedicated section of the AWS data center — completely isolated from other customers. We created one VPC with four subnets across two availability zones.

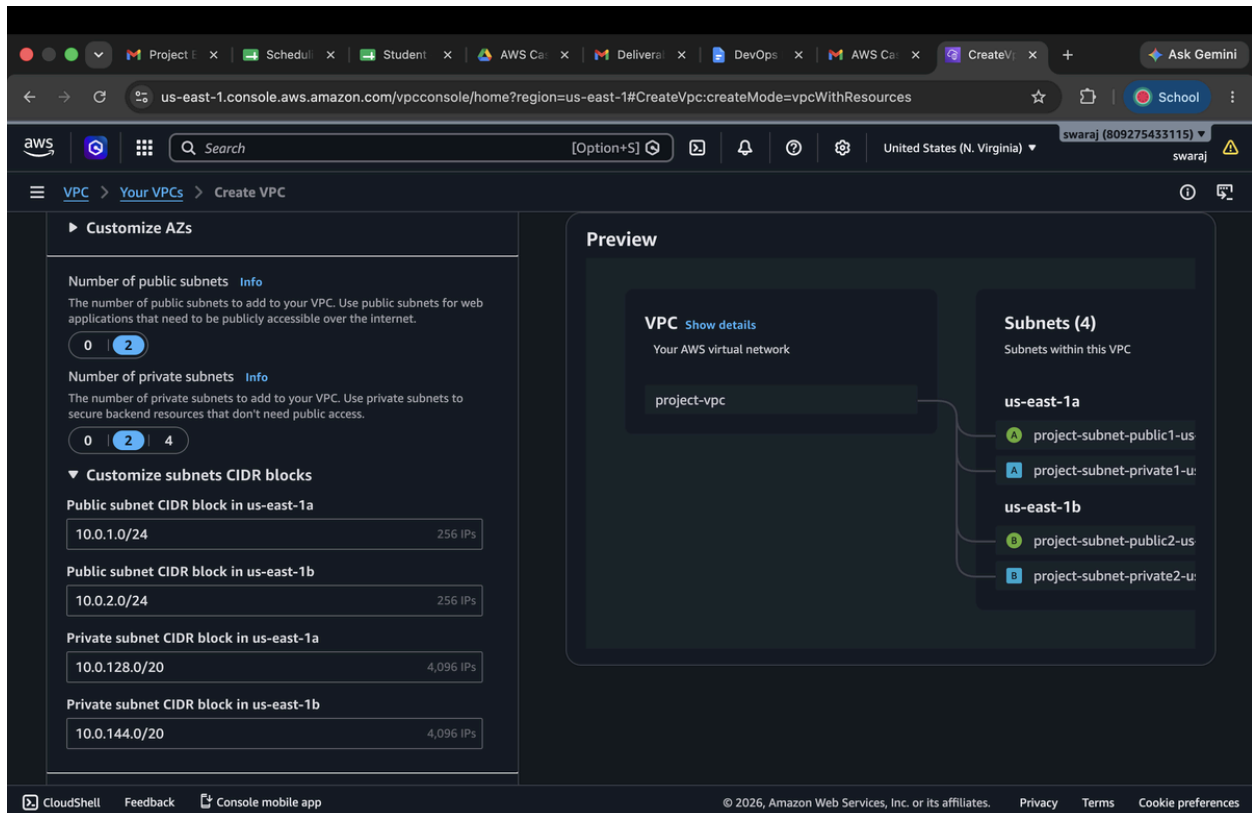


Figure 1 — VPC creation with public and private subnets across two availability zones

4.2 VPC Successfully Created

The VPC was created with all resources auto-provisioned. The state shows "Available" confirming the network is live and ready.

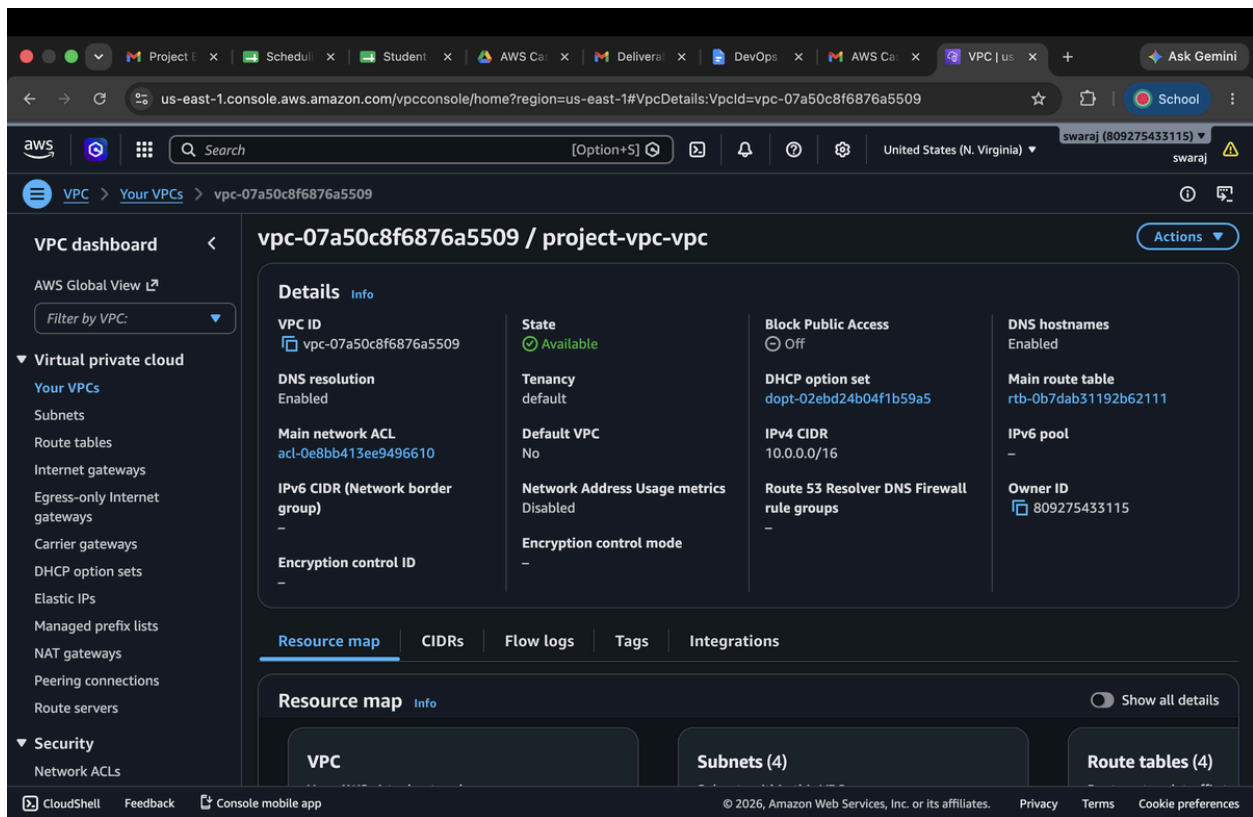


Figure 2 — VPC created with CIDR 10.0.0.0/16, state: Available

4.3 Route Table & Internet Gateway

The route table confirms that public subnet traffic going to the internet (0.0.0.0/0) is directed through the Internet Gateway and is active.

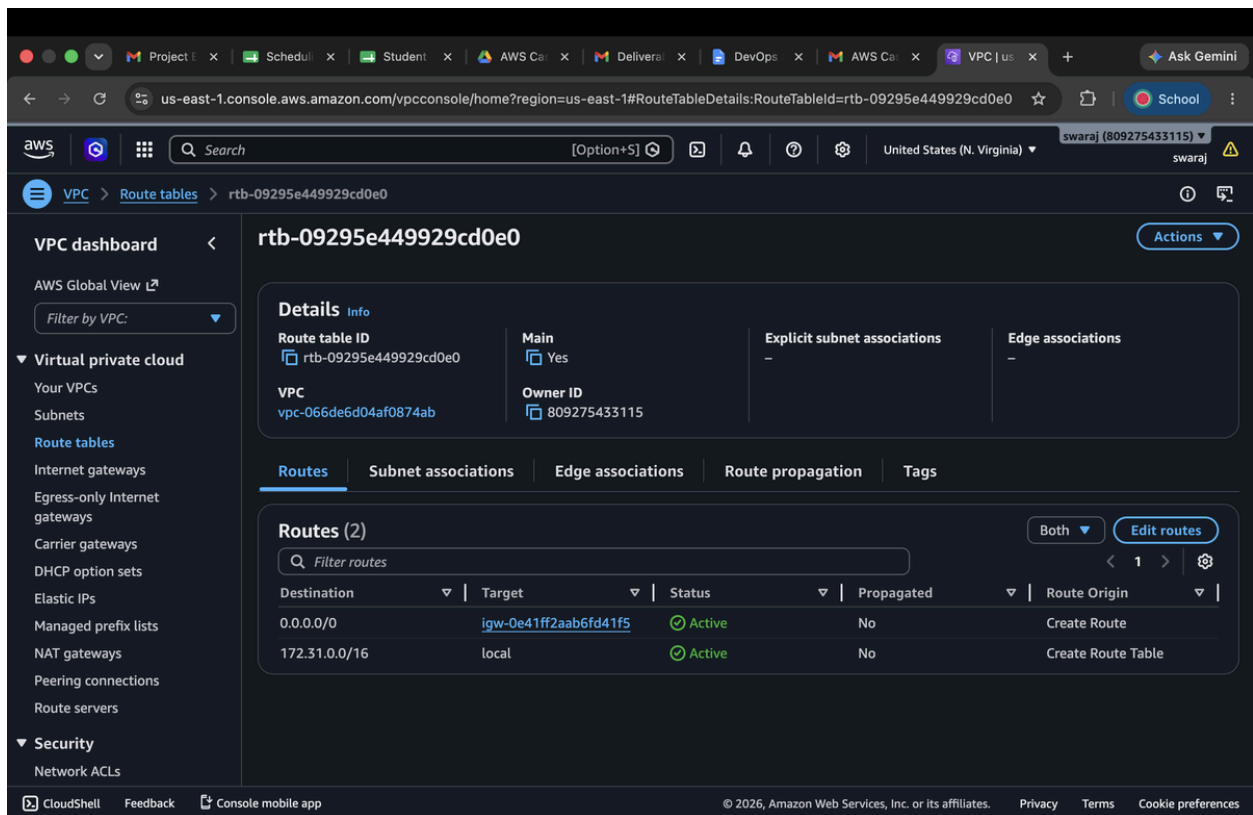


Figure 3 — Route table showing 0.0.0.0/0 → IGW route is Active

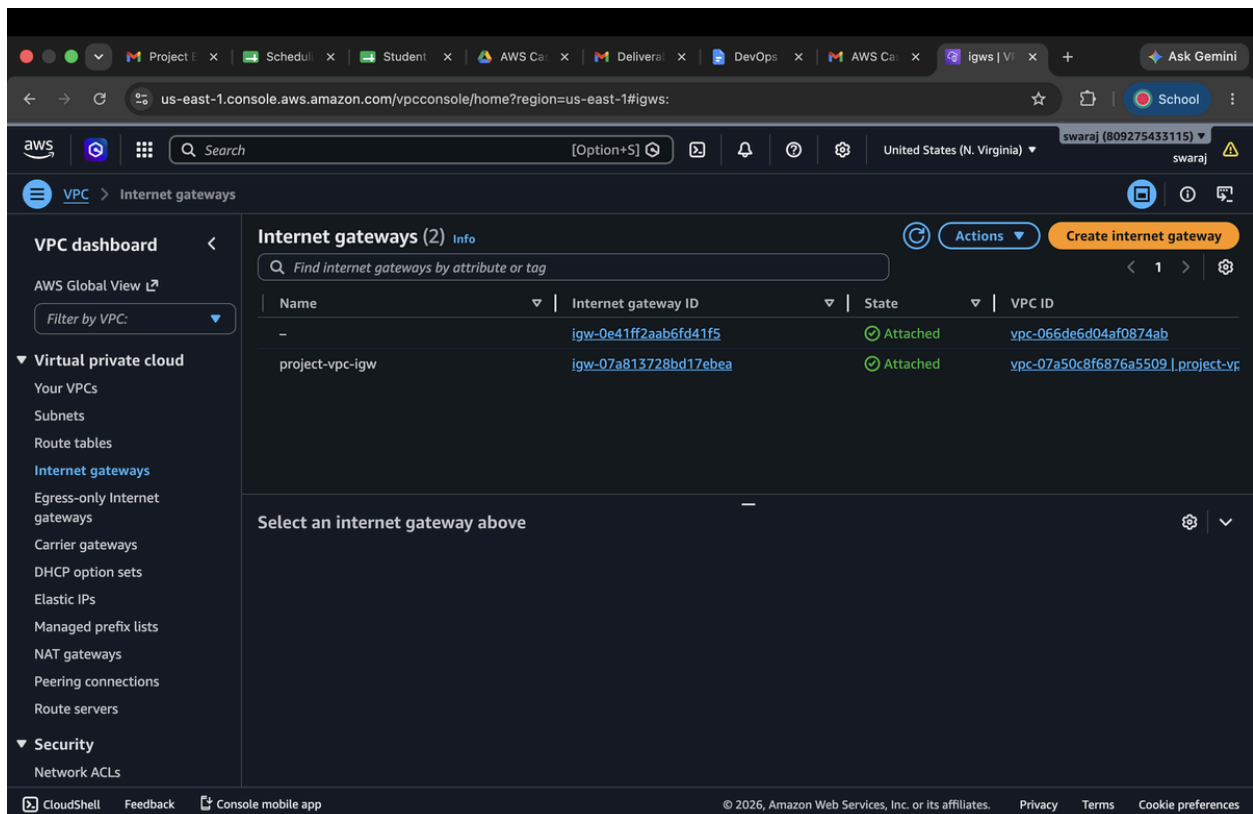


Figure 4 — Internet Gateway "project-vpc-igw" attached and active

4.4 Security Group (Firewall Rules)

We created "wastewise-sg" with three inbound rules: SSH on port 22 (our IP only), HTTP on port 80, and HTTPS on port 443 — so users can access the web application.

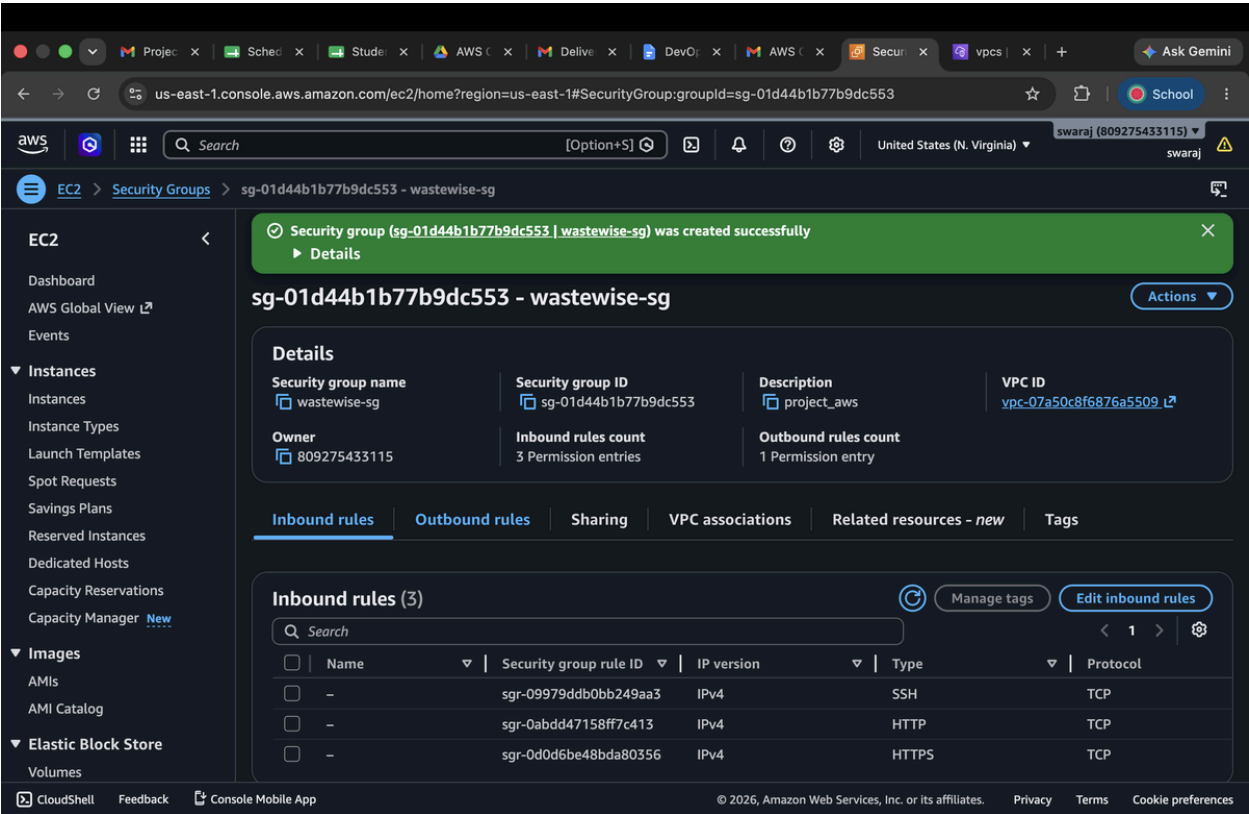


Figure 5 — Security group "wastewise-sg" with 3 inbound rules: SSH, HTTP, HTTPS

4.5 EC2 Server — Running

We launched an EC2 virtual server inside our VPC with a public IP (3.239.60.159) assigned in the public subnet. It runs Amazon Linux 2023.

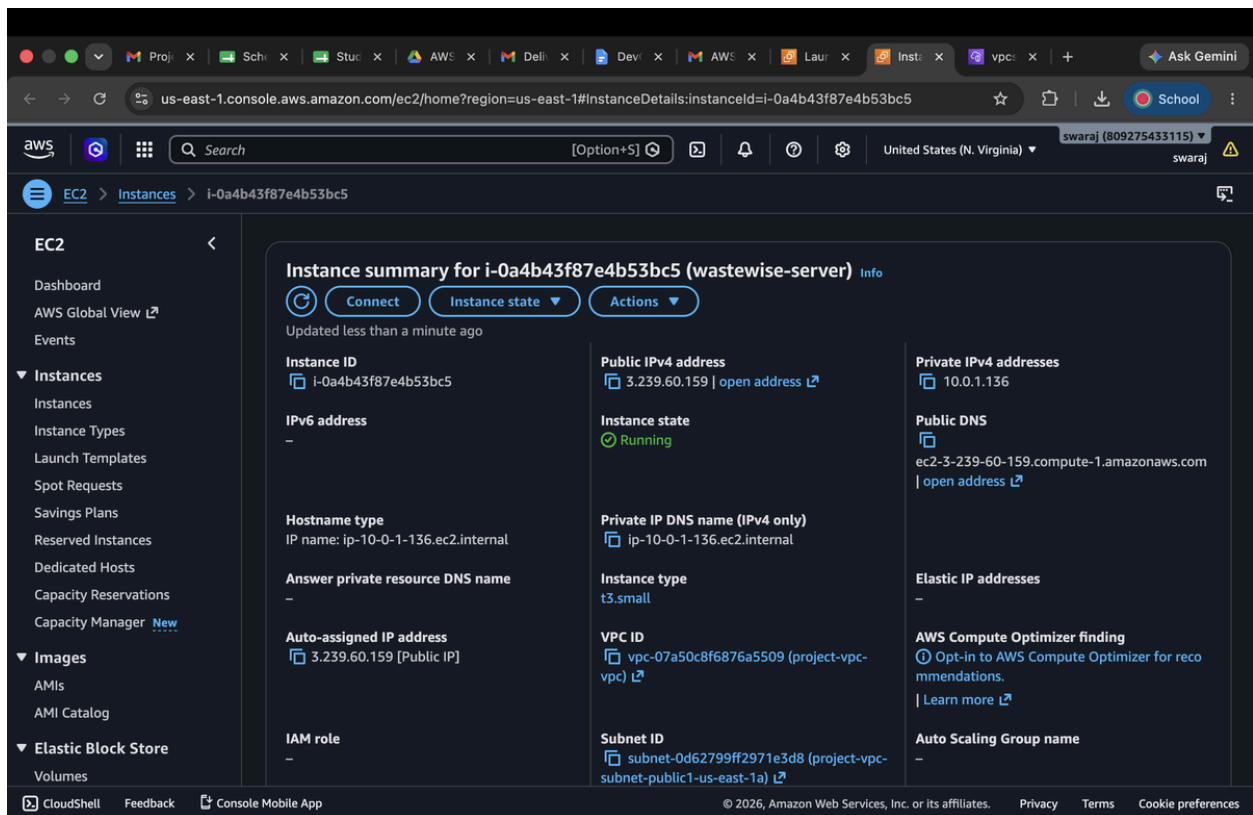


Figure 6 — EC2 instance "wastewise-server" in Running state, Public IP: 3.239.60.159

4.6 Apache Web Server — Live

After SSH-ing into the EC2 server, we installed Apache and started it. The output confirms Apache was installed, started, and enabled to auto-start on every reboot.

```
Complete!
[ec2-user@ip-10-0-1-136 ~]$ sudo systemctl start httpd
[ec2-user@ip-10-0-1-136 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[ec2-user@ip-10-0-1-136 ~]$ ^C
[ec2-user@ip-10-0-1-136 ~]$ exit
logout
Connection to ec2-3-239-60-159.compute-1.amazonaws.com closed.
swarajwattamwar@swarajs-MacBook-Air Downloads %
* History restored
```

Figure 7 — Apache installed, started and enabled on EC2

5. Phase 2 — Database Setup (RDS MySQL)

5.1 RDS Security Group

Before creating the database, we created a separate security group for RDS. This group only allows MySQL traffic (port 3306) from the EC2 security group — the database is completely unreachable from the internet.

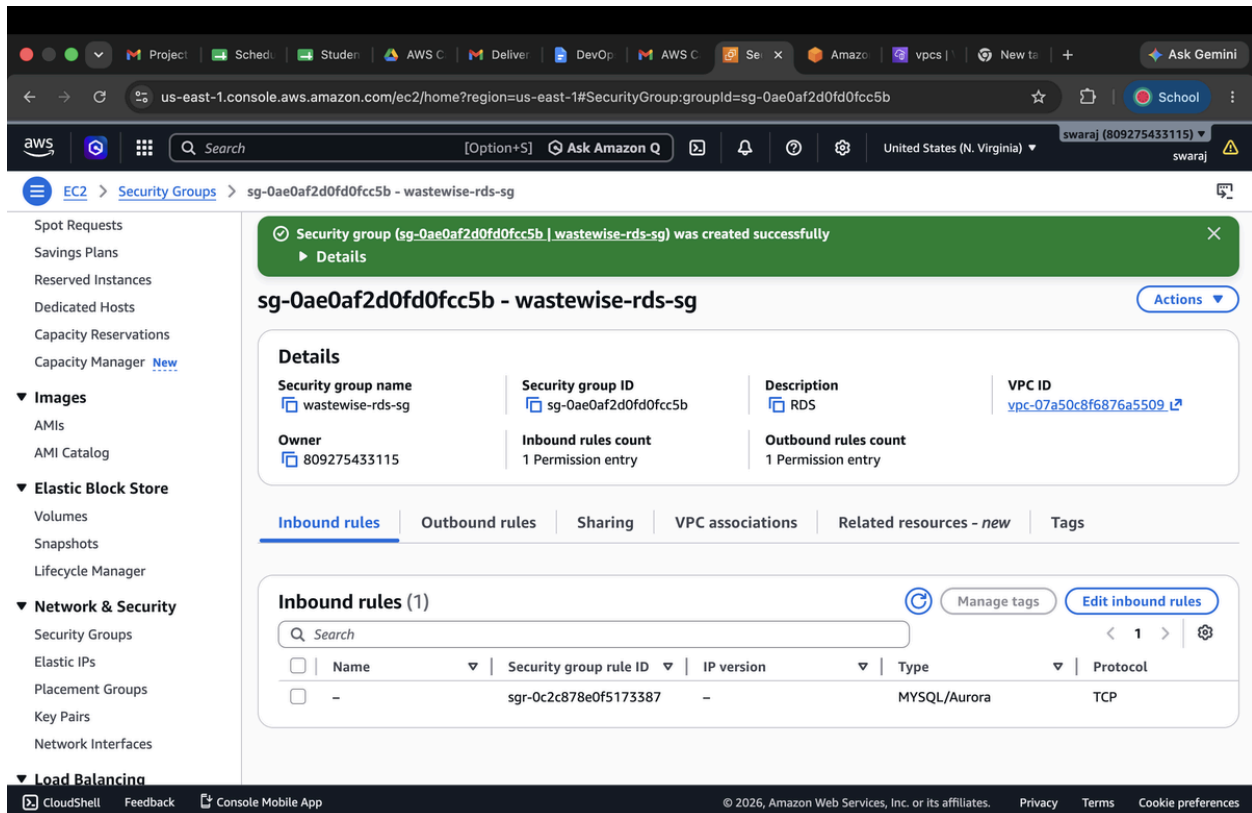


Figure 8 — RDS security group with MySQL/Aurora port 3306 open only from EC2

5.2 RDS Instance Created

"wastewise-db" runs MySQL 8.4.8 on db.t3.micro in the private subnet. Internet Access shows "Disabled" — confirming the database has no public access.

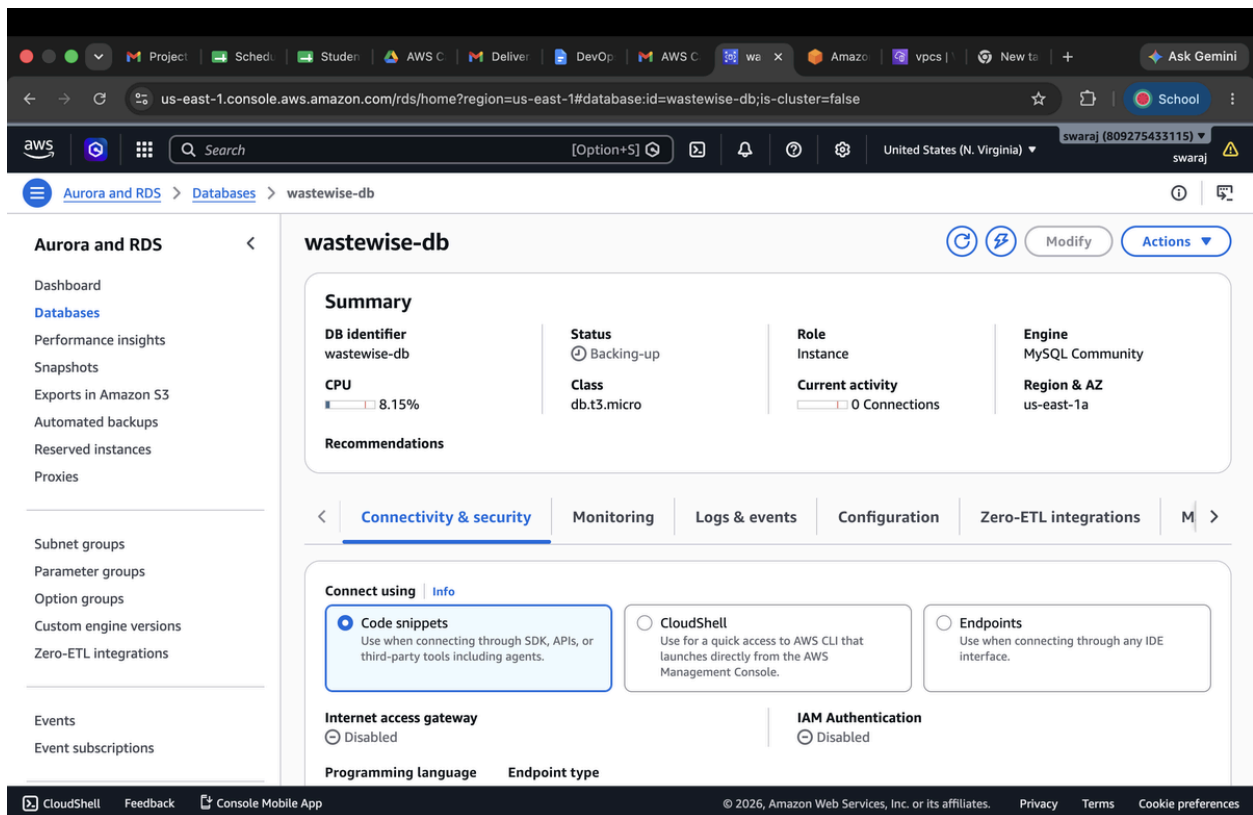


Figure 9 — RDS "wastewise-db" with MySQL 8.4.8, Internet Access Disabled

5.3 Connected to RDS from EC2

We connected to the RDS database from inside EC2 using the MySQL client. The "Welcome to the MariaDB monitor" message and MySQL connection ID confirm successful connection.

```
Complete!
[ec2-user@ip-10-0-1-136 ~]$ wastewise-db.cuhy8a6okev7.us-east-1.rds.amazonaws.com
-bash: wastewise-db.cuhy8a6okev7.us-east-1.rds.amazonaws.com: command not found
[ec2-user@ip-10-0-1-136 ~]$ mysql -h wastewise-db.cuhy8a6okev7.us-east-1.rds.amazonaws.com
ERROR 1045 (28000): Access denied for user 'ec2-user'@'10.0.1.136' (using password: NO)
[ec2-user@ip-10-0-1-136 ~]$ mysql -h wastewise-db.cuhy8a6okev7.us-east-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 33
Server version: 8.4.8 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]>
```

Figure 10 — Successful MySQL connection from EC2 to RDS (MySQL 8.4.8)

5.4 WasteWise Database Tables Created

Two tables were created in the wastewise database: "waste_collections" for tracking collection events, and "vehicles" for fleet management.

```
MySQL [wastewise]> CREATE TABLE vehicles (  
-> id INT AUTO_INCREMENT PRIMARY KEY,  
-> vehicle_id VARCHAR(50),  
-> driver_name VARCHAR(100),  
-> region VARCHAR(100),  
-> capacity_kg DECIMAL(10,2),  
-> status VARCHAR(20)  
-> );
```

Query OK, 0 rows affected (0.027 sec)

```
MySQL [wastewise]> SHOW TABLES;
```

Tables_in_wastewise
vehicles
waste_collections

2 rows in set (0.001 sec)

```
MySQL [wastewise]> 
```

Figure 11 — SHOW TABLES confirming waste_collections and vehicles tables in wastewise DB

6. Phase 3 — Linux Administration

Linux administration covers managing the server — creating users, setting permissions, monitoring processes, reading logs, and scheduling automated tasks. All done directly on the EC2 instance.

6.1 Users & Groups

We created a "wastewise-team" group and two users — "appuser" and "devuser" — both assigned to that group. A dedicated directory at /var/wastewise was secured so only wastewise-team members can access it.

6.2 Cron Job — Automated Scheduling

We installed the cron scheduler and set two automated jobs: a health log every 5 minutes, and a database backup every day at 2 AM. Both confirmed active with crontab -l.

```
Complete!
[ec2-user@ip-10-0-1-136 ~]$ sudo systemctl start crond
[ec2-user@ip-10-0-1-136 ~]$ sudo systemctl enable crond
[ec2-user@ip-10-0-1-136 ~]$ crontab -e
no crontab for ec2-user - using an empty one
crontab: installing new crontab
[ec2-user@ip-10-0-1-136 ~]$ crontab -l
*/5 * * * * /home/ec2-user/scripts/system-log.sh
[ec2-user@ip-10-0-1-136 ~]$
```

Figure 12 — crontab -l showing two scheduled jobs: health log every 5 min, DB backup at 2 AM daily

7. Phase 4 — Docker & Containerization

Docker allows us to package applications into portable containers. Each service runs in its own isolated container — we can start, stop, or update containers without affecting anything else on the server.

7.1 Two Containers Running — Live in Browser

We deployed two Docker containers: "wastewise-web" (nginx from Docker Hub on port 8080) and "wastewise-app" (custom image built from our Dockerfile on port 8081). Both confirmed running in the health check.

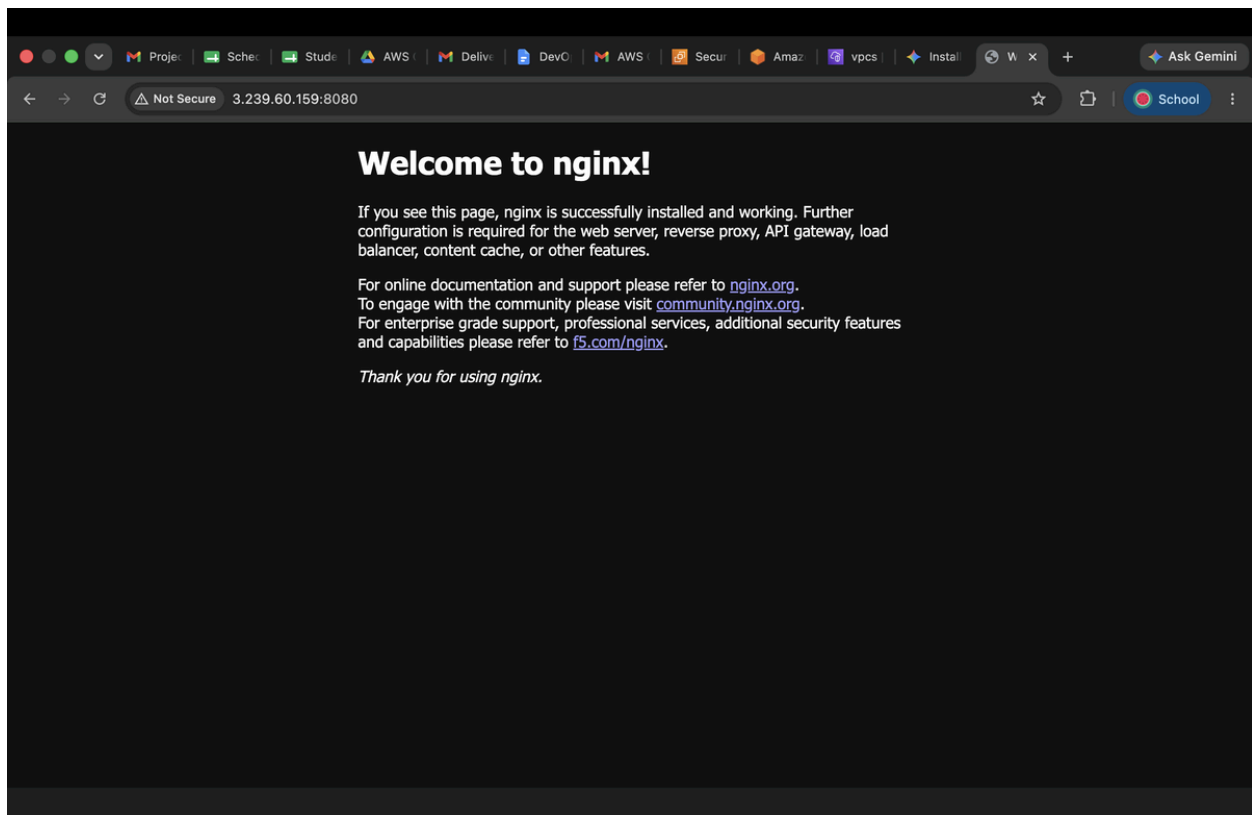


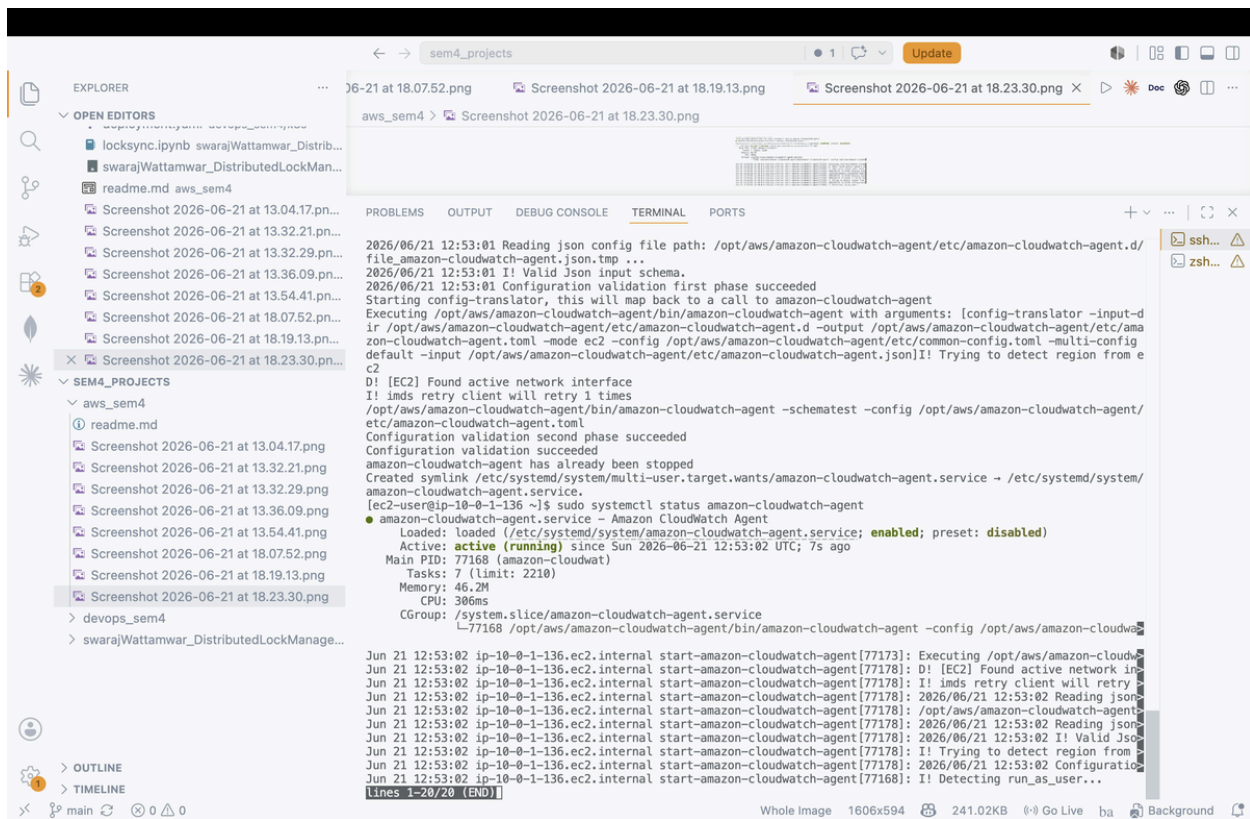
Figure 13 — nginx Docker container live in browser at [http://3.239.60.159:8080](\"http://3.239.60.159:8080\")

8. Phase 5 — Monitoring with CloudWatch

Without monitoring, the team only knows something is wrong when users complain. CloudWatch fixes this by continuously collecting metrics and alerting the team immediately when something needs attention.

8.1 CloudWatch Agent Running

The AWS CloudWatch Agent was installed and configured on the EC2 server. It collects memory usage, disk usage, and custom application logs — metrics that AWS does not track by default. Status shows "active (running)".



```
2026/06/21 12:53:01 Reading json config file path: /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d/
file_amazon-cloudwatch-agent.json.tmp ...
2026/06/21 12:53:01 I! Valid json input schema.
2026/06/21 12:53:01 Configuration validation first phase succeeded
Starting config-translator, this will map back to a call to amazon-cloudwatch-agent
Executing /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent with arguments: [config-translator -input-d
ir /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d -output /opt/aws/amazon-cloudwatch-agent/etc/ama
zon-cloudwatch-agent.toml -mode ec2 -config /opt/aws/amazon-cloudwatch-agent/etc/common-config.toml -multi-config
default -input /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json]! Trying to detect region from e
c2
D! [EC2] Found active network interface
I! imds retry client will retry 1 times
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent -schematest -config /opt/aws/amazon-cloudwatch-agent/
etc/amazon-cloudwatch-agent.toml
Configuration validation second phase succeeded
Configuration validation succeeded
amazon-cloudwatch-agent has already been stopped
Created symlink /etc/systemd/system/multi-user.target.wants/amazon-cloudwatch-agent.service → /etc/systemd/system/
amazon-cloudwatch-agent.service.
[ec2-user@ip-10-0-1-136 ~]$ sudo systemctl status amazon-cloudwatch-agent
● amazon-cloudwatch-agent.service - Amazon CloudWatch Agent
   Loaded: loaded (/etc/systemd/system/amazon-cloudwatch-agent.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-06-21 12:53:02 UTC; 7s ago
   Main PID: 77168 (amazon-cloudwat)
   Tasks: 7 (Limit: 2210)
   Memory: 46.2M
   CPU: 306ms
   CGroup: /system.slice/amazon-cloudwatch-agent.service
           └─77168 /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent -config /opt/aws/amazon-cloudwa

Jun 21 12:53:02 ip-10-0-1-136.ec2.internal start-amazon-cloudwatch-agent[77173]: Executing /opt/aws/amazon-cloudwa
Jun 21 12:53:02 ip-10-0-1-136.ec2.internal start-amazon-cloudwatch-agent[77178]: D! [EC2] Found active network in
Jun 21 12:53:02 ip-10-0-1-136.ec2.internal start-amazon-cloudwatch-agent[77178]: I! imds retry client will retry
Jun 21 12:53:02 ip-10-0-1-136.ec2.internal start-amazon-cloudwatch-agent[77178]: 2026/06/21 12:53:02 Reading json
Jun 21 12:53:02 ip-10-0-1-136.ec2.internal start-amazon-cloudwatch-agent[77178]: /opt/aws/amazon-cloudwatch-agent
Jun 21 12:53:02 ip-10-0-1-136.ec2.internal start-amazon-cloudwatch-agent[77178]: 2026/06/21 12:53:02 Reading json
Jun 21 12:53:02 ip-10-0-1-136.ec2.internal start-amazon-cloudwatch-agent[77178]: 2026/06/21 12:53:02 I! Valid Jso
Jun 21 12:53:02 ip-10-0-1-136.ec2.internal start-amazon-cloudwatch-agent[77178]: I! Trying to detect region from
Jun 21 12:53:02 ip-10-0-1-136.ec2.internal start-amazon-cloudwatch-agent[77178]: 2026/06/21 12:53:02 Configuratio
Jun 21 12:53:02 ip-10-0-1-136.ec2.internal start-amazon-cloudwatch-agent[77168]: I! Detecting run_as_user...
```

Figure 14 — CloudWatch Agent active (running) on EC2, collecting memory, disk and log metrics

CPU Utilization	CloudWatch alarm set — triggers alert if CPU exceeds 80% for 5 minutes
Memory Usage	Collected by CloudWatch Agent (not tracked by AWS by default)
Disk Usage	Collected by CloudWatch Agent — alerts before disk fills up
App Logs	health.log streamed to CloudWatch log group "wastewise-health-logs"
RDS Metrics	DB connections and performance tracked automatically by RDS

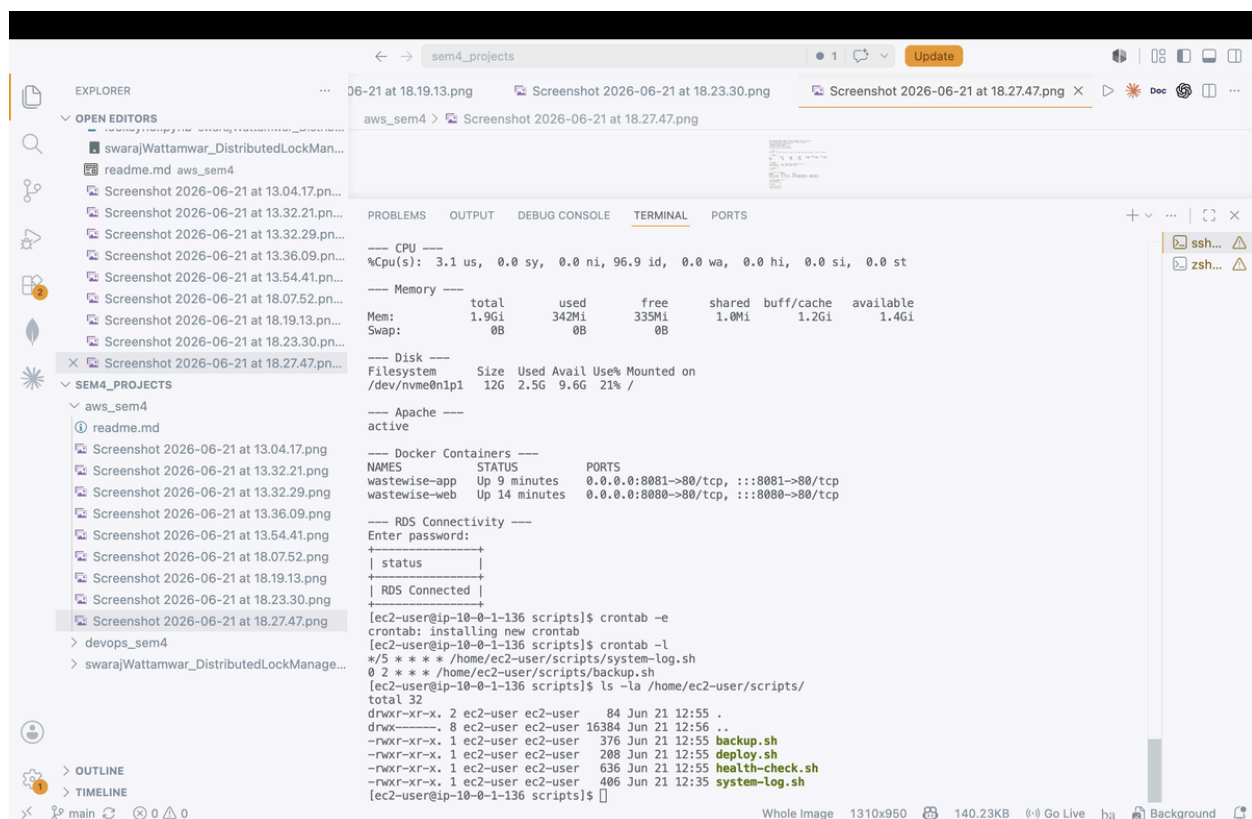
9. Phase 6 — Automation & Shell Scripts

Manual tasks are slow and error-prone. We wrote four shell scripts that automate the most critical recurring tasks. These are scheduled via cron so they run automatically.

backup.sh	Dumps the entire RDS MySQL database to a .sql file — runs every day at 2 AM automatically
deploy.sh	Restarts Apache and Docker containers — one command to push any update
health-check.sh	Prints a full system report: CPU, memory, disk, Apache, Docker containers, RDS connection
system-log.sh	Writes a health snapshot to a log file every 5 minutes via cron

9.1 Health Check — All Systems Green

The screenshot below shows the health-check.sh script output. Every component is confirmed healthy: CPU at 3.1%, memory available, disk at 21%, Apache active, both Docker containers up, and RDS connection successful.



```
sem4_projects
Update

aws_sem4 > Screenshot 2026-06-21 at 18.27.47.png

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

--- CPU ---
%Cpu(s):  3.1 us,  0.0 sy,  0.0 ni, 96.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st

--- Memory ---
Mem:      total      used      free      shared buff/cache available
Swap:      0B          0B          0B          1.0Mi    1.2Gi    1.4Gi

--- Disk ---
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p1 12G  2.5G  9.6G  21% /

--- Apache ---
active

--- Docker Containers ---
NAMES      STATUS      PORTS
wastewise-app Up 9 minutes  0.0.0.0:8081->80/tcp, :::8081->80/tcp
wastewise-web Up 14 minutes 0.0.0.0:8080->80/tcp, :::8080->80/tcp

--- RDS Connectivity ---
Enter password:
+-----+
| status |
+-----+
| RDS Connected |
+-----+

[ec2-user@ip-10-0-1-136 scripts]$ crontab -e
crontab: installing new crontab
[ec2-user@ip-10-0-1-136 scripts]$ crontab -l
*/5 * * * * /home/ec2-user/scripts/system-log.sh
0 2 * * * /home/ec2-user/scripts/backup.sh
[ec2-user@ip-10-0-1-136 scripts]$ ls -la /home/ec2-user/scripts/
total 32
drwxr-xr-x. 2 ec2-user ec2-user 84 Jun 21 12:55 .
drwxr-xr-x. 8 ec2-user ec2-user 16384 Jun 21 12:56 ..
-rwxr-xr-x. 1 ec2-user ec2-user 376 Jun 21 12:55 backup.sh
-rwxr-xr-x. 1 ec2-user ec2-user 208 Jun 21 12:55 deploy.sh
-rwxr-xr-x. 1 ec2-user ec2-user 636 Jun 21 12:55 health-check.sh
-rwxr-xr-x. 1 ec2-user ec2-user 406 Jun 21 12:35 system-log.sh
[ec2-user@ip-10-0-1-136 scripts]$
```

Figure 15 — health-check.sh output: Apache active, 2 Docker containers running, RDS Connected